tending to increase $P_\theta(a|s_t)$, but for a particularly long episode (if we're doing well in the game and get a large end-of-episode return) we will increase the $P_\theta(a|s_t)$ to a greater degree. Hence, on average over many episodes we will reinforce the actions that are good, and the bad actions will get left behind. Since probabilities must sum to 1, if we increase the probability of a good action, that will automatically steal probability mass from the other presumably less good actions. Without this redistributive nature of probabilities, this scheme wouldn't work (i.e., everything both good and bad would tend to increase).

Calculating future rewards

We multiply $P(a_t|\theta,s_t)$ by the total reward (a.k.a. return) we received after this state. As mentioned earlier in the section, we can get the total reward by just summing the rewards (which is equal to the number of time steps the episode lasted in CartPole) and create a return array that starts with the episode duration and decrements by 1 until 1. If the episode lasted 5 time steps, the return array would be `[5,4,3,2,1]`. This makes sense because our first action should be rewarded the most, since it is the least responsible for the pole falling and losing the episode. In contrast, the action right before the pole fell is the worst action, and it should have the smallest reward. But this is a linear decrement—we want to discount the rewards exponentially.

To compute the discounted rewards, we make an array of γ_t by taking our γ parameter, which may be set to 0.99 for example, and exponentiating it according to the distance from the end of the episode. For example, we start with `gamma_t = [0.99, 0.99, 0.99, 0.99, 0.99]`, then create another array of exponents `exp = [1,2,3,4,5]` and compute `torch.power(gamma_t, exp)`, which will give us `[1.0, 0.99, 0.98, 0.97, 0.96]`.

The loss function

Now that we have discounted returns, we can use these to compute the loss function to train the policy network. As we discussed previously, we make our loss function the negative log-probability of the action given the state,